

Structured Outputs = 让大模型按照你规定好的数据结构稳定输出结果，像一个“可控的数据生成器”，而不是随意写一段文本。

1. 它解决什么问题？

普通大模型输出经常有几个问题：

1. 你要求它输出 JSON，它可能漏字段。
2. 你要求字段值只能是某几个枚举值，它可能编一个新的。
3. 你要求某个字段是数组，它可能输出成字符串。
4. 你后端代码要解析模型输出时，经常需要额外校验和重试。

Structured Outputs 的作用就是：

你先定义一个 JSON Schema，模型输出必须严格遵守这个 Schema。

通俗理解：

以前是你跟模型说：“请按这个格式写。”

Structured Outputs 是你给模型一张“表格模板”，它必须按格子填，不能乱加列，也不能漏列。

2. 它的核心好处

文件里总结了几个主要好处：

1) 可靠的类型安全

模型输出会符合你定义的字段类型，比如：

```
{  
  "name": "string",  
  "date": "string",  
  "participants": ["string"]  
}
```

你后端代码就可以放心地用这些字段，而不是先写一堆防御性解析逻辑。

通俗解释：

就像你要求员工填 Excel 表，Structured Outputs 确保每一列都按规定填，不会有人把“日期”填到“参与人”那一列。

2) 安全拒答可以被程序识别

如果用户输入涉及安全问题，模型可能拒绝回答。Structured Outputs 会提供一个 `refusal` 字段，让程序可以识别这是“拒答”，而不是普通输出失败。

这很重要，因为在实际产品里，你需要区分：

- 模型正常返回了结构化结果；
 - 模型因为安全原因拒绝；
 - 模型调用失败；
 - 输出被截断。
-

3) Prompt 可以更简单

以前为了让模型稳定输出 JSON，可能要写很多提示词：

“必须输出 JSON，不要输出解释，不要使用 markdown，不要漏字段……”

Structured Outputs 让这类提示减少很多，因为格式约束交给 Schema 和 API 层完成。

3. Structured Outputs 有两种使用方式

文件区分了两种场景：

A. Function Calling 场景

当你希望模型调用你系统里的工具、函数、数据库、接口时，用 **function calling**。

例如：

- 查询订单；
- 查询数据库；
- 控制 UI；
- 调用搜索工具；
- 调用内部业务系统。

通俗解释：

模型不是直接回答用户，而是决定“我要调用哪个工具、传什么参数”。

比如用户说：

帮我查一下订单 123 的状态。

模型可能输出：

```
{
  "function": "get_order_status",
  "arguments": {
    "order_id": "123"
  }
}
```

B. text.format / response_format 场景

当你希望模型直接给用户返回一个固定结构的答案时，用 **Structured Outputs via text.format**。

例如：

- 数学题分步骤输出；
- 简历分析结果；
- 文章摘要；
- 风险评级；
- 财报提取；
- 用户意图分类；
- UI 渲染所需结构。

通俗解释：

模型还是在回答用户，但回答内容不是一段散文，而是一份结构化数据。

4. Structured Outputs 和 JSON Mode 的区别

这是文件里非常关键的一点。

对比项	Structured Outputs	JSON Mode
是否保证是合法 JSON	是	是
是否保证符合指定 Schema	是	否
是否适合稳定工程化	更适合	需要额外校验
推荐使用	优先使用	作为较基础方案

通俗解释：

JSON Mode 只保证“这是一个合法 JSON”。

Structured Outputs 保证“这是一个合法 JSON，而且长得必须和你定义的模板一模一样”。

比如你想要：

```
{
  "name": "Alice",
  "age": 20
}
```

JSON Mode 可能返回：

```
{
  "username": "Alice",
  "age": "twenty"
}
```

这仍然是合法 JSON，但不符合你的业务结构。

Structured Outputs 则会尽量确保字段名、字段类型、枚举值都符合你的定义。

5. 文件中的示例：提取日历事件

文件给了一个例子：从一句自然语言中提取事件信息。

输入：

Alice and Bob are going to a science fair on Friday.

Schema 规定输出：

```
class CalendarEvent(BaseModel):  
    name: str  
    date: str  
    participants: list[str]
```

模型会输出类似：

```
{  
  "name": "science fair",  
  "date": "Friday",  
  "participants": ["Alice", "Bob"]  
}
```

通俗解释：

这就是“非结构化文本 → 结构化数据”的典型应用。
你可以把一段自然语言变成数据库可以存储、程序可以处理的字段。

这对你的 AI 项目很有价值，比如：

- 从金融新闻里抽取事件；
- 从研报里抽取公司、指标、观点；
- 从会议纪要里抽取任务、负责人、截止日期；
- 从用户问题里抽取意图和参数。

6. 文件中的另一个示例：数学推理步骤

文件还展示了一个数学辅导应用，Schema 包括：

```
class Step(BaseModel):  
    explanation: str  
    output: str  
  
class MathReasoning(BaseModel):  
    steps: list[Step]  
    final_answer: str
```

模型会按步骤输出：

```
{
  "steps": [
    {
      "explanation": "Start with the equation...",
      "output": "8x + 7 = -23"
    }
  ],
  "final_answer": "x = -15 / 4"
}
```

通俗解释：

你可以把模型的回答拆成多个模块，比如“步骤解释”“公式”“最终答案”。
前端可以分别渲染这些模块，而不是只能展示一整段文本。

这背后是一个重要思想：

Structured Outputs 不只是为了程序解析，也是为了产品体验设计。

7. 使用时的三个步骤

文件总结的使用流程是：

Step 1: 定义 Schema

也就是先定义你希望模型输出什么结构。

例如：

```
{
  "name": "string",
  "date": "string",
  "participants": ["string"]
}
```

Step 2: 在 API 调用里传入 Schema

告诉模型：你的输出必须符合这个结构。

Step 3: 处理边界情况

包括：

- 用户输入无关；
- 模型拒答；
- 输出不完整；
- 任务过于复杂；
- Schema 不被支持。

通俗解释：

Schema 能约束输出格式，但不能保证模型理解永远正确。
所以工程上仍然需要设计异常处理。

8. 一个重要提醒：结构正确不等于内容正确

文件特别提醒：Structured Outputs 仍然可能包含错误。

也就是说，它能保证：

```
{  
  "company": "Apple",  
  "revenue": 123  
}
```

这个结构是对的。

但它不能天然保证：

- revenue 数值一定准确；
- company 是否识别正确；
- 因果解释是否真实；
- 信息是否来自可靠来源。

通俗解释：

Structured Outputs 解决的是“格式稳定性”，不是“事实真实性”。

所以在你的金融 Agent 场景里，它应该和这些机制配合使用：

1. 数据源校验；
2. 工具调用；
3. JSON Schema 校验；
4. 引用来源；

5. 置信度字段；
6. 人工复核或自动规则复核。

9. 对用户输入要有兜底设计

文件提醒，如果用户输入和任务完全无关，模型仍然会努力填 Schema，这可能导致幻觉。

比如你要求模型提取会议事件，但用户输入是：

| 今天心情不错。

模型可能为了满足 Schema，硬编一个事件。

所以 prompt 里应该明确规定：

| 如果输入不包含事件信息，则返回空字段或特定说明。

例如可以设计成：

```
{
  "has_event": false,
  "event_name": null,
  "date": null,
  "participants": []
}
```

10. Schema 支持哪些类型？

文件列出了 Structured Outputs 支持的主要 JSON Schema （“格式说明书”，用来强制 AI 输出整齐、标准、能直接用的 JSON 结果）**类型：

- **string** 文本 / 字符串，必须用引号包起来
- **number** 可以是整数，也可以是小数
- **boolean** 布尔（是 / 否）
- **integer** 整数（没有小数点）
- **object** 对象（一组信息打包）：把多个字段包在一起，形成一条完整信息
- **array** 数组（列表）：一组有序的东西，用 `[]` 包起来
- **enum** 枚举（固定选项）：只能从规定好的列表里选一个
- **anyOf** 满足其中任意一种就行，多个类型任选其一，可以是数字，也可以是字符串

也支持一些约束，比如：

- 字符串格式：email、date、uuid、ipv4 等；
- 数字范围：minimum、maximum；
- 数组长度：minItems、maxItems；
- 正则匹配：pattern；
- 枚举值：enum。

通俗解释：

你不仅可以规定字段是什么类型，还可以规定它必须满足什么条件。

例如：

```
{
  "username": {
    "type": "string",
    "pattern": "^@[a-zA-Z0-9_]+$"
  }
}
```

意思是用户名必须以 @ 开头。

11. Schema 的一些限制

文件也列出了不少限制，比较重要的包括：

1) 根对象必须是 object

最外层不能直接是 anyOf 。

也就是说，Schema 最外层应该像这样：

```
{
  "type": "object",
  "properties": {}
}
```

而不是一上来就是多个分支选择。

2) 所有字段都必须 required

Structured Outputs 要求所有字段都列在 `required` 里。

如果你想表达“可选字段”，可以用 `null` 来模拟：

```
{
  "unit": {
    "type": ["string", "null"],
    "enum": ["F", "C", null]
  }
}
```

通俗解释：

字段不能“有没有都行”，而是必须出现。
但字段值可以是 `null`，表示没有。

3) 对象必须设置 `additionalProperties: false`

这表示模型不能输出 Schema 以外的字段。

例如你规定只有：

```
{
  "location": "string",
  "unit": "string"
}
```

它就不能额外输出：

```
{
  "confidence": 0.9
}
```

除非你在 Schema 里明确规定了 `confidence` 字段。

通俗解释：

不能让模型“自由发挥加字段”，否则后端就不稳定。

4) 嵌套和字段数量有限制

文件提到：

- 最多约 5000 个对象属性；
- 最多 10 层嵌套；
- enum 数量和字符串总长度也有限制。

这对普通应用基本够用，但如果你设计特别复杂的 Schema，就需要拆分任务。

12. Streaming 也可以配合 Structured Outputs

文件提到，Structured Outputs 可以和 streaming 一起用。

这意味着模型生成结构化数据时，你不必等完整结果出来，可以边生成边处理。

适合场景：

- 前端逐字段展示；
- 长报告生成；
- Agent 工具参数流式解析；
- 用户体验要求更实时的应用。

通俗解释：

就像视频边下载边播放，结构化结果也可以边生成边解析。

13. 它对你做 Agent / AI 工程的意义

结合你现在做 AI Agent、金融市场异动监控、研报生成等方向，这份文件的价值很大。

Structured Outputs 可以用于这些场景：

1) 意图识别

把用户问题变成结构化意图：

```
{  
  "intent": "query_market_anomaly",  
  "asset_class": "equity",  
  "region": "HK",
```

```
"time_range": "last_week"
}
```

2) 新闻事件抽取

从新闻中提取：

```
{
  "company": "string",
  "event_type": "earnings_warning",
  "impact_direction": "negative",
  "confidence": "medium"
}
```

3) 异常归因卡片

你之前的 Market Insight Agent 可以输出：

```
{
  "fact": "港股成交额环比上升",
  "magnitude": "较上周上升 15%",
  "possible_drivers": ["南向资金流入", "政策预期升温"],
  "confidence": "medium"
}
```

4) 报告渲染

先让模型生成结构化 JSON，再由代码渲染成 Word、网页或仪表盘。

这比直接让模型写完整报告更稳定。

14. 最重要的工程化启发

这份文件真正想传达的不是“怎么输出 JSON”，而是：

大模型应用要从“文本生成”转向“结构化接口”。

也就是说，大模型不再只是聊天机器人，而是系统中的一个模块。

它的输入、输出、错误、拒答、字段、类型，都应该被工程化约束。

对 AI Agent 来说，这非常关键：

没有 Structured Outputs	使用 Structured Outputs
模型自由写文本	模型按 Schema 输出
后端难解析	后端稳定读取
容易漏字段	字段强约束
Prompt 很脆弱	Schema 更可靠
难以接入自动流程	易接入 workflow、数据库、前端

总结

这份文件可以概括为四句话：

- 1. Structured Outputs 是 OpenAI 用来保证模型输出符合 JSON Schema 的能力。
- 2. 它比 JSON Mode 更强，因为 JSON Mode 只保证合法 JSON，不保证符合 Schema。
- 3. 它适合构建稳定的 AI 应用、Agent、数据抽取、报告生成和前端 UI 渲染。
- 4. 它解决的是格式和接口稳定性，但不自动保证事实正确，仍需配合校验、工具调用和异常处理。

15、如何开发 Schema

第一阶段：探索型 Schema

这个阶段不要追求完美，只定义最核心字段。

比如金融市场异动解释，一开始可以只要：

```
{
  "summary": "string",
  "key_facts": ["string"],
  "possible_drivers": ["string"],
  "confidence": "low | medium | high"
}
```

这个 Schema 很粗，但足够让你开始观察模型会怎么组织信息。

它的目标不是“准确建模业务”，而是：

先把模型输出从散文变成半结构化卡片。

第二阶段：稳定型 Schema

当你跑了几十个样本后，会发现哪些字段经常有用，哪些字段经常混乱。

比如你可能发现：

- `possible_drivers` 太宽泛，需要区分“已验证驱动”和“推测驱动”；
- `confidence` 不够，需要加 `evidence_level`；
- `summary` 太模糊，需要拆成 `fact`、`magnitude`、`impact`；
- 模型经常把原因说得太确定，需要加 `causal_claim_level`。

这时候 Schema 可以演化成：

```
{
  "fact": "string",
  "magnitude": "string",
  "impact": "string",
  "possible_drivers": [
    {
      "driver": "string",
      "evidence": "string",
      "confidence": "low | medium | high"
    }
  ],
  "uncertainties": ["string"]
}
```

这个阶段开始体现业务判断。

第三阶段：生产型 Schema

等你的下游用途明确了，比如要渲染成周报、进数据库、做前端卡片、触发告警，Schema 才需要更严格。

例如：

```
{
  "asset_class": "equity | bond | fx | commodity",
  "region": "CN | HK | US | Global",
  "metric_name": "string",
  "change_direction": "up | down | flat",
  "change_magnitude": "number",
```

```
"change_unit": "pct | bp | amount | index_point",
"severity_level": "low | medium | high",
"narrative": "string",
"drivers": [
  {
    "driver_type": "macro | policy | liquidity | earnings | valuation |
technical | unknown",
    "description": "string",
    "evidence_strength": "weak | moderate | strong"
  }
],
"requires_human_review": "boolean"
}
```

这个阶段才适合严格接入系统。

所以 Schema 不是一次性设计，而是版本化演进

你可以把它当成数据库表结构、API contract 或 data contract。

一开始字段不全很正常，关键是要有版本意识：

```
schema_v0.1: 能跑通
schema_v0.2: 补充常见字段
schema_v0.3: 处理异常和空值
schema_v1.0: 下游系统稳定依赖
```

每次改 Schema，都问三个问题：

1. 这个字段是下游真的要用，还是我觉得“好像有用”？
2. 这个字段模型能不能稳定产出？
3. 这个字段如果为空、错误或不确定，系统怎么处理？

一个实用原则：先宽后严

早期不要把 Schema 设计得太细，否则你会被结构困住。

比如不要一开始就设计 30 个字段。可以先用 5 到 8 个字段，让模型多跑样本。

比较好的早期字段通常是：

```
{
  "task_understood": "boolean",
  "summary": "string",
  "structured_points": ["string"],
```

```
"missing_information": ["string"],
"confidence": "low | medium | high",
"needs_schema_revision": "boolean"
}
```

这里最关键的是 `missing_information` 和 `needs_schema_revision`。

它们的作用是让模型帮你发现：

这个任务当前 Schema 装不下什么信息？

这就把 Schema 设计从“拍脑袋”变成了“通过样本迭代”。

最推荐你的做法：双轨输出

在项目早期，可以让模型同时输出两类内容：

一类是严格结构化字段，给程序用；

另一类是自由解释区，给人看、给你迭代 Schema 用。

例如：

```
{
  "structured_result": {
    "summary": "string",
    "key_facts": ["string"],
    "confidence": "low | medium | high"
  },
  "schema_feedback": {
    "missing_fields": ["string"],
    "ambiguous_fields": ["string"],
    "suggested_new_fields": ["string"]
  },
  "freeform_notes": "string"
}
```

这样做的好处是：

既保留 Structured Outputs 的稳定性，又不牺牲早期探索空间。

对你的结论

你的判断是对的：

一开始字段可能不全，也可能有误，所以 Structured Outputs 不适合一上来就做成很重、很死的 Schema。

更合理的使用方式是：

把 Schema 当成一个逐步成熟的接口契约，而不是一开始就必须完美的设计文档。

在 AI 应用开发里，Schema 不是前置一次性工作，而是和 prompt、样本、错误案例、下游需求一起迭代出来的工程资产。

16、如何把 AI 变成一个稳定可接入系统的模块

AI 的贡献不在于“帮你省掉所有设计”，而在于它把过去很难工程化的部分变得可工程化。

1. 工程负责“边界”，AI 负责“判断与生成”

可以这样理解：

层次	主要由谁负责	作用
JSON Schema	工程 / 产品 / 业务人员	规定输出边界
Prompt	人	规定任务目标和判断标准
模型	AI	理解输入、抽取信息、做分类、生成解释
校验与渲染	工程系统	确保结果可用、可展示、可存储

所以 Structured Outputs 不是让 AI 替代工程设计，而是让 AI 的结果能被工程系统稳定接住。

通俗说：

Schema 是轨道，AI 是车。
没有轨道，车可以到处跑，但很难进入生产系统；
没有 AI，轨道再清楚，也没人能自动理解复杂文本、做判断、生成内容。

文件里也强调，Structured Outputs 的价值在于让模型输出遵守你提供的 JSON Schema，避免漏字段、乱枚举、格式不稳定等问题；但它并不等于事实一定正确，仍需要处理拒答、错误、用户输入不匹配等边界情况。

2. AI 的真正贡献：处理“非结构化 → 结构化”

这件事以前非常难。

比如一段金融新闻：

港股今日大涨，主要受政策预期升温、南向资金流入以及科技股反弹带动。

传统工程代码很难稳定判断：

- 这是市场上涨还是下跌？
- 涉及哪个市场？
- 驱动因素有哪些？
- 哪些是事实，哪些是推测？
- 该不该进入周报？
- 应该如何写成一段人能读懂的解释？

AI 的贡献就是：它可以理解自然语言，把复杂、模糊、非结构化的信息转成结构化结果。

例如：

```
{
  "market": "港股",
  "direction": "up",
  "drivers": ["政策预期升温", "南向资金流入", "科技股反弹"],
  "claim_type": "possible_explanation",
  "confidence": "medium"
}
```

工程系统擅长处理“规则清楚的数据”；

AI 擅长处理“含义复杂的文本”。

Structured Outputs 的作用是把两者接起来。

3. AI 不是减少所有工作，而是改变工作分工

你原来可能期待的是：

| AI 能不能直接把事情做完？

但在工程系统里，更现实的答案是：

| AI 负责不确定性部分，工程负责确定性部分。

举个例子。

不适合交给 AI 的部分

这些更应该由工程规则做：

- 字段名称；
- 字段类型；
- 数据库表结构；
- 报告模板；

- 是否允许空值；
- 枚举值范围；
- 接口调用顺序；
- 失败重试机制。

适合交给 AI 的部分

这些才是 AI 的强项：

- 从长文本中提取核心信息；
- 判断一句话属于哪个业务类型；
- 归纳多个原因；
- 生成解释性文字；
- 判断信息是否充分；
- 给出可能驱动；
- 把专家经验转成初步标签；
- 对边界案例给出人类可读解释。

所以 AI 的价值不是“不要工程”，而是：

把原来必须靠人阅读、判断、归纳、表达的部分，变成可批量处理的能力。

4. Schema 反而让 AI 更有价值

看起来 Schema 限制了 AI，但实际上它让 AI 更容易进入真实业务流程。

如果没有 Schema，AI 输出可能是：

今天港股表现较好，可能与政策预期和资金流入有关，科技股也有一定贡献。

这句话人能看懂，但程序很难用。

有 Schema 后，AI 输出变成：

```
{
  "fact": "港股表现较好",
  "possible_drivers": ["政策预期", "资金流入", "科技股反弹"],
  "certainty": "possible",
  "reportable": true
}
```

这时系统就可以继续做：

- 入库；

- 排序；
- 生成图表；
- 渲染周报；
- 触发人工复核；
- 和历史数据对比；
- 给前端不同模块展示。

也就是说：

Schema 不是削弱 AI，而是**让 AI 的输出从“可读”变成“可用”。**

5. AI 还可以反过来帮助你设计 Schema

你刚才提到“一开始字段未必想清楚”，这正是 AI 可以参与的地方。

AI 不一定只是 Schema 的执行者，也可以是 Schema 的共创者。

比如你可以给 AI 一批样本，让它帮你总结：

这些金融新闻中，哪些信息是反复出现、值得结构化的？
哪些字段适合作为稳定字段？
哪些字段只适合作为自由文本？
哪些字段容易混淆？
哪些字段应该做成 enum？

AI 可以帮你产出初版字段建议：

```
{
  "recommended_fields": [
    "asset_class",
    "market",
    "direction",
    "magnitude",
    "possible_drivers",
    "evidence_level",
    "uncertainties"
  ],
  "fields_to_avoid_initially": [
    "precise_causal_weight",
    "single_definitive_reason"
  ],
  "reason": "多数样本只能支持可能归因，无法支持精确因果拆分。"
}
```

这就是 AI 在工程设计前期的贡献：

它可以帮助你从样本中发现结构，而不只是被动填结构。

6. 最准确的理解：AI 负责“语义层”，工程负责“契约层”

我建议你整个系统分成两层：

契约层：工程确定性

这层回答：

系统需要什么格式？

包括 Schema、字段、类型、接口、校验、版本管理。

语义层：AI 不确定性

这层回答：

这段信息是什么意思？

包括理解、抽取、分类、归因、总结、解释。

Structured Outputs 的本质就是：

用工程的契约层，承接 AI 的语义层。

这也是为什么它对 Agent 很重要。Agent 真正难的不是让模型“说一段话”，而是让模型在系统里稳定地产生下一步可执行的信息。